



6 LUGLIO 2026

Documentazione dell'implementazione Java + JML del sistema ascensore

MAFFEIS RICCARDO 1085706



Sommario

1. Introduzione.....	2
2. Differenze rispetto al modello ASM	3
3. Struttura del codice.....	4
4. Classe Ascensore.....	5
5. Rappresentazione delle richieste attive	6
6. Invarianti JML della classe.....	6
6.1 Vincoli sullo stato numerico.....	6
6.2 Vincoli di non nullità.....	7
6.3 Invarianti di sicurezza sul movimento.....	7
6.4 Invarianti sul guasto	7
6.5 Invarianti sul sovraccarico.....	7
6.6 Invariante sul timer.....	7
7. Costruttore e stato iniziale	8
8. Metodi principali	9
8.1 Metodi di osservazione.....	9
8.2 Gestione delle richieste.....	10
8.3 Gestione persone e sovraccarico	11
8.4 Gestione guasto e timer	13
8.5 Gestione porte, servizio del piano e stato di attesa.....	15
8.6 Ricerca richieste, scelta direzione e movimento	16
9. Classe InputAscensore	19
10. Classe ControlloreAscensore	20
11. Interfaccia web dimostrativa	21
11.1 Modalità automatica	22
11.2 Modalità manuale	23
12. Test JUnit e Test Parametrici	24
12.1 Collegamento tra scenari AVALLA e test JUnit.....	25
12.2 Conversione test Avalla creati con ATGT in JUnit.....	25
13. Test Selenium.....	26
14. Verifica tramite ESC/OpenJML	27
15. Conclusione.....	28

1. Introduzione

Questa documentazione descrive l'implementazione Java del nucleo logico del sistema ascensore modellato precedentemente mediante ASM.

L'obiettivo dell'implementazione Java non è sostituire il modello ASM, ma realizzare una versione eseguibile e verificabile del comportamento principale dell'ascensore.

La classe *Ascensore* rappresenta il nucleo operativo del sistema ed è annotata mediante JML per specificare formalmente proprietà, vincoli e contratti dei metodi principali.

Alla classe *Ascensore* si affiancano la classe *InputAscensore*, usata per rappresentare gli input esterni del sistema, e la classe *ControlloreAscensore*, che coordina l'evoluzione dell'ascensore a ogni passo logico.

La verifica statica tramite ESC/OpenJML consente di controllare la coerenza del codice rispetto alle proprietà dichiarate, mentre i test JUnit 5 verificano il comportamento operativo del nucleo Java in scenari concreti.

A questi si aggiungono test parametrici JUnit 5, utilizzati per controllare in modo più sistematico classi di input, valori limite e condizioni di validità.

Oltre al nucleo logico, il progetto include una semplice interfaccia web dimostrativa basata su *HttpServer*, utilizzata per osservare dinamicamente il comportamento dell'ascensore e per interagire con il sistema in modalità automatica o manuale. L'interfaccia utilizza direttamente le classi del nucleo Java e non introduce una nuova logica di controllo.

La parte web è stata inoltre verificata tramite test Selenium, che controllano l'integrazione tra pagina web, server HTTP e nucleo logico Java.

2. Differenze rispetto al modello ASM

L'implementazione Java organizza la logica del sistema in metodi separati.

Ogni metodo rappresenta una parte del comportamento del modello ASM.

Nel modello ASM gli input esterni erano rappresentati da funzioni monitorate:

```
// Input esterni
monitored chiamataPianoSu: Piano -> Boolean
monitored chiamataPianoGiu: Piano -> Boolean
monitored pulsanteInterno: Piano -> Boolean
monitored eventoGuasto: Boolean
monitored personeEntrate: Integer
monitored personeUscite: Integer
```

Nel codice Java questi input sono raccolti nella classe *InputAscensore* e poi interpretati dal *ControlloreAscensore*.

Il controllore usa poi tali informazioni per invocare i metodi della classe *Ascensore*.

Inoltre, mentre nel modello ASM il comportamento complessivo è coordinato da una regola principale, nella versione Java tale ruolo è svolto dal metodo *eseguiPasso(InputAscensore input)* della classe *ControlloreAscensore*.

3. Struttura del codice

Il codice è organizzato principalmente nel package progetto, che contiene il nucleo logico del sistema. A questo si aggiungono il package web, dedicato al server HTTP dell'interfaccia dimostrativa, la cartella webapp, che contiene la parte statica dell'interfaccia, e il package test, dedicato ai test automatici.

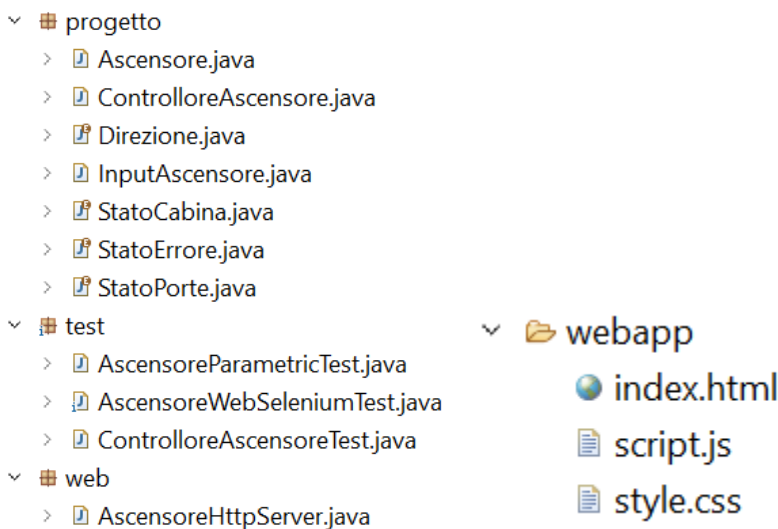
Le classi principali sono:

- *Ascensore*, che contiene lo stato interno e i metodi fondamentali dell'ascensore;
- *InputAscensore*, che rappresenta gli input esterni ricevuti in un passo logico;
- *ControlloreAscensore*, che coordina l'evoluzione del sistema;
- *AscensoreHttpServer*, che realizza il server HTTP dell'interfaccia web dimostrativa;
- *ControlloreAscensoreTest*, che contiene i test JUnit 5 funzionali sul nucleo logico;
- *AscensoreParametricTest*, che contiene test parametrici JUnit 5 su classi di input e casi limite;
- *AscensoreWebSeleniumTest*, che contiene i test Selenium sull'interfaccia web.

La parte statica dell'interfaccia è separata in tre file:

- *index.html*, che definisce la struttura della pagina;
- *style.css*, che definisce lo stile grafico;
- *script.js*, che contiene la logica JavaScript di aggiornamento e interazione.

Le enumerazioni utilizzate sono Direzione, StatoCabina, StatoPorte e StatoErrore.



4. Classe Ascensore

La classe *Ascensore* contiene lo stato principale del sistema e i metodi che implementano la logica dell'ascensore.

Le costanti principali sono:

```
public static final int PIANO_MINIMO = -1;
public static final int PIANO_MASSIMO = 4;
public static final int NUMERO_PIANI = 6;
public static final int CAPACITA_MASSIMA = 8;
public static final int TIMER_MASSIMO = 10;
```

Queste costanti definiscono:

- il piano minimo raggiungibile;
- il piano massimo raggiungibile;
- il numero totale di piani;
- la capacità massima della cabina;
- il valore massimo del timer di ripristino dopo un guasto.

Essendo dichiarate come `public static final`, tali costanti sono fissate direttamente dal linguaggio Java e non necessitano di ulteriori invarianti JML.

Lo stato interno dell'ascensore è rappresentato dai seguenti attributi:

```
private /*@ spec_public @*/ int pianoCorrente;
private /*@ spec_public @*/ int timer;
private /*@ spec_public @*/ int numeroPersone;

private /*@ spec_public @*/ StatoCabina statoCabina;
private /*@ spec_public @*/ StatoPorte statoPorte;
private /*@ spec_public @*/ Direzione direzione;
private /*@ spec_public @*/ StatoErrore statoErrore;

private /*@ spec_public @*/ final boolean[] richiesteAttive;
```

Gli attributi sono dichiarati `private` per proteggere lo stato interno della classe e impedire modifiche dirette non controllate dall'esterno.

L'annotazione JML `/*@ spec_public @*/` consente comunque di usare questi campi nelle specifiche JML pubbliche.

5. Rappresentazione delle richieste attive

Nel modello ASM le richieste erano rappresentate come funzione:

```
controlled richiesteAttive: Piano -> Boolean
```

Nel codice Java sono rappresentate tramite un array:

```
private /*@ spec_public @*/ final boolean[] richiesteAttive;
```

Poiché in Java gli array partono da indice 0, mentre i piani dell'ascensore vanno da -1 a 4, viene utilizzato il metodo `indiceDelPiano(int piano)` che converte il numero del piano nell'indice dell'array.

Il contratto JML del metodo è:

```
/*@ requires pianoValido(piano);  
/*@ ensures 0 <= \result && \result < NUMERO_PIANI;  
/*@ ensures \result == piano - PIANO_MINIMO;  
private /*@ spec_public @*/ /*@ pure @*/ int indiceDelPiano(int piano) {  
    return piano - PIANO_MINIMO;  
}
```

Questo garantisce che ogni piano valido venga convertito in un indice valido dell'array.

6. Invarianti JML della classe

La classe `Ascensore` definisce diversi invarianti JML. Gli invarianti rappresentano proprietà che devono essere sempre vere per ogni oggetto valido della classe.

6.1 Vincoli sullo stato numerico

```
~  
@ public invariant PIANO_MINIMO <= pianoCorrente && pianoCorrente <= PIANO_MASSIMO;  
@ public invariant 0 <= timer && timer <= TIMER_MASSIMO;  
@ public invariant numeroPersone >= 0;
```

Questi invarianti garantiscono che:

- il piano corrente sia sempre compreso tra il piano minimo e il piano massimo;
- il timer sia sempre compreso tra 0 e `TIMER_MASSIMO`;
- il numero di persone non sia mai negativo.

6.2 Vincoli di non nullità

```
@ public invariant statoCabina != null;
@ public invariant statoPorte != null;
@ public invariant direzione != null;
@ public invariant statoErrore != null;
@
@ public invariant richiesteAttive != null;
@ public invariant richiesteAttive.length == NUMERO_PIANI;
```

Questi invarianti garantiscono che gli stati dell'ascensore siano sempre definiti e che l'array delle richieste sia correttamente inizializzato.

6.3 Invarianti di sicurezza sul movimento

```
@ public invariant statoCabina == StatoCabina.IN_MOVIMENTO ==> statoPorte == StatoPorte.CHIUSE;
@ public invariant statoCabina == StatoCabina.IN_MOVIMENTO ==> statoErrore == StatoErrore.NESSUNO;
```

Questi invarianti formalizzano due proprietà fondamentali:

- la cabina può muoversi solo con le porte chiuse;
- la cabina può muoversi solo se non sono presenti errori.

6.4 Invarianti sul guasto

```
@ public invariant statoErrore == StatoErrore.GUASTO ==> statoCabina == StatoCabina.BLOCCATA;
@ public invariant statoErrore == StatoErrore.GUASTO ==> statoPorte == StatoPorte.CHIUSE;
@ public invariant statoErrore == StatoErrore.GUASTO ==> direzione == Direzione.NESSUNA;
```

Quando il sistema è in stato GUASTO, la cabina deve essere bloccata, le porte devono essere chiuse e la direzione deve essere annullata.

6.5 Invarianti sul sovraccarico

```
@ public invariant statoErrore == StatoErrore.OVERLOAD ==> statoCabina == StatoCabina.BLOCCATA;
@ public invariant statoErrore == StatoErrore.OVERLOAD ==> statoPorte == StatoPorte.APERTE;
@ public invariant statoErrore == StatoErrore.OVERLOAD ==> direzione == Direzione.NESSUNA;
```

Quando il sistema è in stato OVERLOAD, la cabina deve essere bloccata, le porte devono essere aperte e la direzione deve essere annullata.

6.6 Invariante sul timer

```
@ public invariant timer > 0 ==> statoErrore == StatoErrore.GUASTO;
```

Questo invariante stabilisce che il timer può essere positivo solo durante uno stato di guasto.

7. Costruttore e stato iniziale

Il costruttore inizializza l'ascensore nello stato iniziale e le postcondizioni JML garantiscono che l'oggetto venga creato in uno stato coerente:

```
//@ ensures pianoCorrente == 0;
//@ ensures timer == 0;
//@ ensures numeroPersone == 0;
//@ ensures statoCabina == StatoCabina.FERMA;
//@ ensures statoPorte == StatoPorte.CHIUSE;
//@ ensures direzione == Direzione.NESSUNA;
//@ ensures statoErrore == StatoErrore.NESSUNO;
//@ ensures richiesteAttive != null;
//@ ensures richiesteAttive.length == NUMERO_PIANI;
public Ascensore() {
    this.pianoCorrente = 0;
    this.timer = 0;
    this.numeroPersone = 0;
    this.statoCabina = StatoCabina.FERMA;
    this.statoPorte = StatoPorte.CHIUSE;
    this.direzione = Direzione.NESSUNA;
    this.statoErrore = StatoErrore.NESSUNO;
    this.richiesteAttive = new boolean[NUMERO_PIANI];
}
```

8. Metodi principali

8.1 Metodi di osservazione

La classe contiene diversi metodi *pure*, utilizzati per leggere lo stato senza modificarlo:

```
//@ ensures \result == pianoCorrente;
public /*@ pure @*/ int getPianoCorrente() {
    return pianoCorrente;
}

//@ ensures \result == timer;
public /*@ pure @*/ int getTimer() {
    return timer;
}

//@ ensures \result == numeroPersone;
public /*@ pure @*/ int getNumeroPersone() {
    return numeroPersone;
}

//@ ensures \result == statoCabina;
public /*@ pure @*/ StatoCabina getStatoCabina() {
    return statoCabina;
}

//@ ensures \result == statoPorte;
public /*@ pure @*/ StatoPorte getStatoPorte() {
    return statoPorte;
}

//@ ensures \result == direzione;
public /*@ pure @*/ Direzione getDirezione() {
    return direzione;
}

//@ ensures \result == statoErrore;
public /*@ pure @*/ StatoErrore getStatoErrore() {
    return statoErrore;
}
```

In JML, un metodo dichiarato *pure* non modifica lo stato dell'oggetto e può quindi essere utilizzato all'interno delle specifiche.

8.2 Gestione delle richieste

La classe permette di aggiungere tre tipi di richieste, le quali richiedono che il piano sia valido e che il sistema non sia in stato di guasto:

```
//@ requires pianoValido(piano);
//@ requires statoErrore != StatoErrore.GUASTO;
//@ ensures richiesteAttive[indiceDelPiano(piano)] == true;
public void aggiungiRichiestaInterna(int piano) {
    richiesteAttive[indiceDelPiano(piano)] = true;
}

//@ requires pianoValido(piano);
//@ requires piano < PIANO_MASSIMO;
//@ requires statoErrore != StatoErrore.GUASTO;
//@ ensures richiesteAttive[indiceDelPiano(piano)] == true;
public void aggiungiChiamataSalita(int piano) {
    richiesteAttive[indiceDelPiano(piano)] = true;
}

//@ requires pianoValido(piano);
//@ requires piano > PIANO_MINIMO;
//@ requires statoErrore != StatoErrore.GUASTO;
//@ ensures richiesteAttive[indiceDelPiano(piano)] == true;
public void aggiungiChiamataDiscesa(int piano) {
    richiesteAttive[indiceDelPiano(piano)] = true;
}
```

Questi metodi rappresentano rispettivamente:

- la pressione di un pulsante interno alla cabina;
- una chiamata esterna verso l'alto;
- una chiamata esterna verso il basso.

8.3 Gestione persone e sovraccarico

Il metodo `aggiornaPersone()` aggiorna il numero di persone presenti in cabina:

```
/*@
@ requires personeEntrate >= 0;
@ requires personeUscite >= 0;
@ requires statoPorte == StatoPorte.APERTE;
@ requires statoCabina == StatoCabina.FERMA || statoCabina == StatoCabina.BLOCCATA;
@ ensures numeroPersone >= 0;
@ ensures \old(numeroPersone) + personeEntrate - personeUscite >= 0 ==>
    numeroPersone == \old(numeroPersone) + personeEntrate - personeUscite;
@ ensures \old(numeroPersone) + personeEntrate - personeUscite < 0 ==>
    numeroPersone == 0;
@*/
@CodeBigintMath
public void aggiornaPersone(int personeEntrate, int personeUscite) {
    int nuovoNumeroPersone = numeroPersone + personeEntrate - personeUscite;

    if (nuovoNumeroPersone >= 0) {
        numeroPersone = nuovoNumeroPersone;
    } else {
        numeroPersone = 0;
    }
}
```

Le precondizioni richiedono che il numero di persone può essere modificato solo se le porte sono aperte e la cabina è ferma o bloccata.

La postcondizione garantisce che il numero di persone non diventi mai negativo.

Il metodo è annotato con `@CodeBigintMath`: questa annotazione viene usata per evitare falsi allarmi legati all'overflow degli interi durante la verifica statica con ESC.

Il metodo `gestisciSovraccarico()` gestisce il passaggio allo stato OVERLOAD:

```
/*@
@ ensures \old(numeroPersone) > CAPACITA_MASSIMA ==>
@   statoErrore == StatoErrore.OVERLOAD &&
@   statoCabina == StatoCabina.BLOCCATA &&
@   statoPorte == StatoPorte.APERTE &&
@   direzione == Direzione.NESSUNA &&
@   timer == 0;
@
@ ensures \old(numeroPersone) <= CAPACITA_MASSIMA &&
@   \old(statoErrore) == StatoErrore.OVERLOAD ==>
@   statoErrore == StatoErrore.NESSUNO &&
@   statoCabina == StatoCabina.FERMA &&
@   statoPorte == StatoPorte.CHIUSE &&
@   direzione == Direzione.NESSUNA &&
@   timer == 0;
@
@ ensures \old(numeroPersone) <= CAPACITA_MASSIMA &&
@   \old(statoErrore) != StatoErrore.OVERLOAD ==>
@   statoErrore == \old(statoErrore) &&
@   statoCabina == \old(statoCabina) &&
@   statoPorte == \old(statoPorte) &&
@   direzione == \old(direzione) &&
@   timer == \old(timer);
@*/
public void gestisciSovraccarico() {
    if (numeroPersone > CAPACITA_MASSIMA) {
        statoErrore = StatoErrore.OVERLOAD;
        statoCabina = StatoCabina.BLOCCATA;
        statoPorte = StatoPorte.APERTE;
        direzione = Direzione.NESSUNA;
        timer = 0;
    } else if (statoErrore == StatoErrore.OVERLOAD) {
        statoErrore = StatoErrore.NESSUNO;
        statoCabina = StatoCabina.FERMA;
        statoPorte = StatoPorte.CHIUSE;
        direzione = Direzione.NESSUNA;
        timer = 0;
    }
}
```

Se il numero di persone supera la capacità massima, il sistema entra in sovraccarico.

Se invece il sistema era in OVERLOAD e il numero di persone torna entro la capacità massima, il sistema ritorna operativo:

8.4 Gestione guasto e timer

Il metodo `attivaGuasto()` attiva lo stato di guasto tecnico:

```
//@ requires statoErrore == StatoErrore.NESSUNO;  
//@ ensures statoErrore == StatoErrore.GUASTO;  
//@ ensures statoCabina == StatoCabina.BLOCCATA;  
//@ ensures statoPorte == StatoPorte.CHIUSE;  
//@ ensures direzione == Direzione.NESSUNA;  
//@ ensures timer == TIMER_MASSIMO;  
public void attivaGuasto() {  
    statoErrore = StatoErrore.GUASTO;  
    statoCabina = StatoCabina.BLOCCATA;  
    statoPorte = StatoPorte.CHIUSE;  
    direzione = Direzione.NESSUNA;  
    timer = TIMER_MASSIMO;  
}
```

Le precondizioni richiedono che il sistema non sia già in errore.

Il metodo `gestisciTimerGuasto()` viene utilizzato quando il sistema si trova nello stato GUASTO:

```
/*@
  @ requires statoErrore == StatoErrore.GUASTO;
  @ ensures \old(timer) > 0 ==> timer == \old(timer) - 1;
  @ ensures \old(timer) > 0 ==> statoErrore == StatoErrore.GUASTO;
  @ ensures \old(timer) == 0 ==>
  @   statoErrore == StatoErrore.NESSUNO &&
  @   statoCabina == StatoCabina.FERMA &&
  @   statoPorte == StatoPorte.CHIUSE &&
  @   direzione == Direzione.NESSUNA &&
  @   timer == 0;
  @*/
public void gestisciTimerGuasto() {
    if (timer > 0) {
        timer = timer - 1;
    } else {
        statoErrore = StatoErrore.NESSUNO;
        statoCabina = StatoCabina.FERMA;
        statoPorte = StatoPorte.CHIUSE;
        direzione = Direzione.NESSUNA;
        timer = 0;
    }
}
```

Il contratto JML descrive il caso in cui il timer è maggiore di zero e quindi viene decrementato e il caso in cui il timer è uguale a zero e di conseguenza il sistema viene ripristinato

8.5 Gestione porte, servizio del piano e stato di attesa

Il metodo `chiudiPorte()` chiude le porte dell'ascensore:

```
//@ requires statoErrore == StatoErrore.NESSUNO;  
//@ requires statoCabina == StatoCabina.FERMA;  
//@ requires statoPorte == StatoPorte.APERTE;  
//@ ensures statoPorte == StatoPorte.CHIUSE;  
public void chiudiPorte() {  
    statoPorte = StatoPorte.CHIUSE;  
}
```

Quindi le porte vengono chiuse solo se il sistema è operativo, la cabina è ferma e le porte sono aperte.

Il metodo `serviPianoCorrente()` serve la richiesta associata al piano corrente:

```
//@ requires statoErrore == StatoErrore.NESSUNO;  
//@ requires richiestaAttiva(pianoCorrente);  
//@ ensures statoCabina == StatoCabina.FERMA;  
//@ ensures statoPorte == StatoPorte.APERTE;  
//@ ensures direzione == Direzione.NESSUNA;  
//@ ensures richiesteAttive[indiceDelPiano(pianoCorrente)] == false;  
public void serviPianoCorrente() {  
    statoCabina = StatoCabina.FERMA;  
    statoPorte = StatoPorte.APERTE;  
    richiesteAttive[indiceDelPiano(pianoCorrente)] = false;  
    direzione = Direzione.NESSUNA;  
}
```

Il metodo rappresenta il momento in cui l'ascensore arriva al piano richiesto, apre le porte e rimuove la richiesta attiva.

Il metodo `mettiInAttesa()` porta l'ascensore nello stato di attesa:

```
//@ requires statoErrore == StatoErrore.NESSUNO;  
//@ ensures statoCabina == StatoCabina.FERMA;  
//@ ensures direzione == Direzione.NESSUNA;  
public void mettiInAttesa() {  
    statoCabina = StatoCabina.FERMA;  
    direzione = Direzione.NESSUNA;  
}
```

Questo metodo rappresenta il comportamento dell'ascensore quando non ci sono richieste da servire.

8.6 Ricerca richieste, scelta direzione e movimento

La classe contiene tre metodi per analizzare le richieste attive.

Il metodo `esisteRichiesta()` verifica se esiste almeno una richiesta attiva in qualsiasi piano:

```
//@ ensures \result <==> (\exists int i; 0 <= i && i < NUMERO_PIANI; richiesteAttive[i]);
public /*@ pure @*/ boolean esisteRichiesta() {
    //@ loop_invariant 0 <= i && i <= NUMERO_PIANI;
    //@ loop_invariant (\forall int j; 0 <= j && j < i; richiesteAttive[j] == false);
    for (int i = 0; i < NUMERO_PIANI; i++) {
        if (richiesteAttive[i]) {
            return true;
        }
    }
    return false;
}
```

Il metodo `esisteRichiestaSopra()` verifica se esiste una richiesta attiva a un piano superiore rispetto al piano corrente:

```
/*@
@ ensures \result <==>
@ (\exists int p; pianoCorrente < p && p <= PIANO_MASSIMO;
@ richiesteAttive[indiceDelPiano(p)]);
@*/
public /*@ pure @*/ boolean esisteRichiestaSopra() {
    int piano = pianoCorrente + 1;

    /*@
    @ loop_invariant pianoCorrente + 1 <= piano && piano <= PIANO_MASSIMO + 1;
    @ loop_invariant (\forall int p; pianoCorrente < p && p < piano;
    @ richiesteAttive[indiceDelPiano(p)] == false);
    @*/
    while (piano <= PIANO_MASSIMO) {
        if (richiesteAttive[indiceDelPiano(piano)]) {
            return true;
        }
        piano++;
    }
    return false;
}
```

Il metodo `esisteRichiestaSotto()` verifica se esiste una richiesta attiva a un piano inferiore rispetto al piano corrente:

```

/*@
@ ensures \result <==>
@   (\exists int p; PIANO_MINIMO <= p && p < pianoCorrente;
@   richiesteAttive[indiceDelPiano(p)]);
@*/
public /*@ pure @*/ boolean esisteRichiestaSotto() {
    int piano = pianoCorrente - 1;

    /*@
    @ loop_invariant PIANO_MINIMO - 1 <= piano && piano <= pianoCorrente - 1;
    @ loop_invariant (\forall int p; piano < p && p < pianoCorrente;
    @   richiesteAttive[indiceDelPiano(p)] == false);
    @*/
    while (piano >= PIANO_MINIMO) {
        if (richiesteAttive[indiceDelPiano(piano)]) {
            return true;
        }

        piano--;
    }

    return false;
}

```

Il metodo `scegliDirezione()` decide la direzione dell'ascensore in base alle richieste attive:

```

/*@
@ requires statoErrore == StatoErrore.NESSUNO;
@ ensures !esisteRichiesta() ==> direzione == Direzione.NESSUNA;
@ ensures \old(direzione) == Direzione.SU && \old(esisteRichiestaSopra()) ==> direzione == Direzione.SU;
@ ensures \old(direzione) == Direzione.GIU && \old(esisteRichiestaSotto()) ==> direzione == Direzione.GIU;
@*/
public void scegliDirezione() {
    if (!esisteRichiesta()) {
        direzione = Direzione.NESSUNA;
        return;
    }

    if (direzione == Direzione.SU) {
        if (esisteRichiestaSopra()) {
            direzione = Direzione.SU;
        } else if (esisteRichiestaSotto()) {
            direzione = Direzione.GIU;
        } else {
            direzione = Direzione.NESSUNA;
        }
    } else if (direzione == Direzione.GIU) {
        if (esisteRichiestaSotto()) {
            direzione = Direzione.GIU;
        } else if (esisteRichiestaSopra()) {
            direzione = Direzione.SU;
        } else {
            direzione = Direzione.NESSUNA;
        }
    } else {
        if (esisteRichiestaSopra()) {
            direzione = Direzione.SU;
        } else if (esisteRichiestaSotto()) {
            direzione = Direzione.GIU;
        } else {
            direzione = Direzione.NESSUNA;
        }
    }
}
}

```

La logica implementata è la seguente:

- se non esistono richieste, la direzione diventa NESSUNA;
- se la direzione corrente è SU e ci sono richieste sopra, la direzione resta SU;
- se la direzione corrente è SU ma non ci sono richieste sopra, il sistema cerca richieste sotto;
- se la direzione corrente è GIU e ci sono richieste sotto, la direzione resta GIU;
- se la direzione corrente è GIU ma non ci sono richieste sotto, il sistema cerca richieste sopra;
- se la direzione corrente è NESSUNA, il sistema sceglie una direzione in base alle richieste disponibili.

Il metodo `muoviDiUnPiano()` sposta l'ascensore di un piano nella direzione corrente:

```
/*  
  @ requires statoErrore == StatoErrore.NESSUNO;  
  @ requires statoPorte == StatoPorte.CHIUSE;  
  @  
  @ ensures PIANO_MINIMO <= pianoCorrente && pianoCorrente <= PIANO_MASSIMO;  
  @  
  @ ensures \old(direzione) == Direzione.SU && \old(pianoCorrente) < PIANO_MASSIMO ==>  
  @   pianoCorrente == \old(pianoCorrente) + 1 &&  
  @   statoCabina == StatoCabina.IN_MOVIMENTO;  
  @  
  @ ensures \old(direzione) == Direzione.GIU && \old(pianoCorrente) > PIANO_MINIMO ==>  
  @   pianoCorrente == \old(pianoCorrente) - 1 &&  
  @   statoCabina == StatoCabina.IN_MOVIMENTO;  
  @  
  @ ensures \old(direzione) == Direzione.SU && \old(pianoCorrente) == PIANO_MASSIMO ==>  
  @   statoCabina == StatoCabina.FERMA &&  
  @   direzione == Direzione.NESSUNA;  
  @  
  @ ensures \old(direzione) == Direzione.GIU && \old(pianoCorrente) == PIANO_MINIMO ==>  
  @   statoCabina == StatoCabina.FERMA &&  
  @   direzione == Direzione.NESSUNA;  
  @*/  
public void muoviDiUnPiano() {  
    if (direzione == Direzione.SU && pianoCorrente < PIANO_MASSIMO) {  
        pianoCorrente = pianoCorrente + 1;  
        statoCabina = StatoCabina.IN_MOVIMENTO;  
    } else if (direzione == Direzione.GIU && pianoCorrente > PIANO_MINIMO) {  
        pianoCorrente = pianoCorrente - 1;  
        statoCabina = StatoCabina.IN_MOVIMENTO;  
    } else {  
        statoCabina = StatoCabina.FERMA;  
        direzione = Direzione.NESSUNA;  
    }  
}
```

Se la direzione è SU e il piano corrente è minore del piano massimo, il piano viene incrementato.

Se la direzione è GIU e il piano corrente è maggiore del piano minimo, il piano viene decrementato.

Se non è possibile muoversi, la cabina viene fermata e la direzione viene annullata.

9. Classe InputAscensore

La classe *InputAscensore* rappresenta gli input esterni ricevuti dal sistema durante un singolo passo logico

Al suo interno vengono memorizzate eventuali richieste interne, chiamate esterne di salita o discesa, l'evento di guasto e il numero di persone entrate o uscite dalla cabina.

Sono inoltre presenti metodi di supporto che permettono di verificare se una richiesta è effettivamente presente:

```
public boolean haRichiestaInterna() {  
    return richiestaInterna != NESSUNA_RICHIESTA;  
}  
  
public boolean haChiamataSalita() {  
    return chiamataSalita != NESSUNA_RICHIESTA;  
}  
  
public boolean haChiamataDiscesa() {  
    return chiamataDiscesa != NESSUNA_RICHIESTA;  
}
```

La classe *InputAscensore* non modifica direttamente lo stato dell'ascensore. Il suo compito è raccogliere gli input esterni, che vengono poi interpretati dalla classe *ControlloreAscensore*.

10. Classe *ControlloreAscensore*

La classe *ControlloreAscensore* ha il compito di coordinare l'evoluzione del sistema a ogni passo logico. Mentre la classe *Ascensore* contiene lo stato interno e i metodi che modificano tale stato, il controllore stabilisce l'ordine con cui questi metodi devono essere invocati.

Questa separazione permette di mantenere distinta la rappresentazione dello stato dalla logica di coordinamento, rendendo il codice più leggibile e più vicino alla struttura del modello ASM, nel quale una regola principale coordina l'esecuzione delle singole regole.

Il metodo principale della classe è

```
public void eseguiPasso(InputAscensore input) {  
    if (input == null) {  
        throw new IllegalArgumentException("InputAscensore non puo essere null");  
    }  
  
    acquisisciRichieste(input);  
    gestisciPersone(input);  
    gestisciErrore(input);  
    gestisciComportamentoNormale();  
}
```

11. Interfaccia web dimostrativa

Oltre al nucleo logico del sistema, il progetto include una semplice interfaccia web dimostrativa realizzata tramite `HttpServer` di Java.

L'interfaccia web ha lo scopo di rendere osservabile il comportamento del nucleo Java dell'ascensore.

La classe `AscensoreHttpServer` espone tre endpoint principali:

- `/`: utilizzato per restituire la pagina HTML dell'interfaccia;
- `/stato`: utilizzato per ottenere lo stato corrente dell'ascensore in formato JSON;
- `/azione`: utilizzato per inviare comandi al sistema, come richieste interne, chiamate esterne, aggiornamento del numero di persone, guasto, reset e gestione della simulazione automatica.

L'interfaccia supporta due modalità operative:

- simulazione automatica;
- controllo manuale.

11.1 Modalità automatica

Nella modalità automatica il sistema genera periodicamente e in modo casuale richieste, ingressi e uscite di persone e possibili guasti.

Simulatore Ascensore

Simulazione automatica

Controllo manuale

Vista ascensore



Stato ascensore

Piano corrente 0	Timer 0
Numero persone 0	Stato cabina FERMA
Stato porte CHIUSE	Direzione NESSUNA
Stato errore NESSUNO	Simulazione automatica FERMA

Comandi rapidi

Avvia simulazione

Ferma simulazione

Reset

Richieste attive

Piano 4	Nessuna
Piano 3	Nessuna
Piano 2	Nessuna
Piano 1	Nessuna
Piano 0	Nessuna
Piano -1	Nessuna

Ultimi eventi

[19:22:25] Sistema inizializzato

11.2 Modalità manuale

Nella modalità manuale l'utente può invece inserire direttamente richieste interne, chiamate esterne, variazioni del numero di persone e guasti.

Simulatore Ascensore

Simulazione automaticaControllo manuale

Vista ascensore



Stato ascensore

Piano corrente 0	Timer 0
Numero persone 0	Stato cabina FERMA
Stato porte CHIUSE	Direzione NESSUNA
Stato errore NESSUNO	Simulazione automatica FERMA

Comandi rapidi

Esegui passoAttiva guastoReset

Richieste attive

Piano 4	Nessuna
Piano 3	Nessuna
Piano 2	Nessuna
Piano 1	Nessuna
Piano 0	Nessuna
Piano -1	Nessuna

Ultimi eventi

[19:26:48] Sistema inizializzato

Richiesta interna

Piano: 0Invia richiesta interna

Rapido: -101234

Chiamata esterna

Piano salita: 0Chiamata salita

Piano discesa: 0Chiamata discesa

Persone

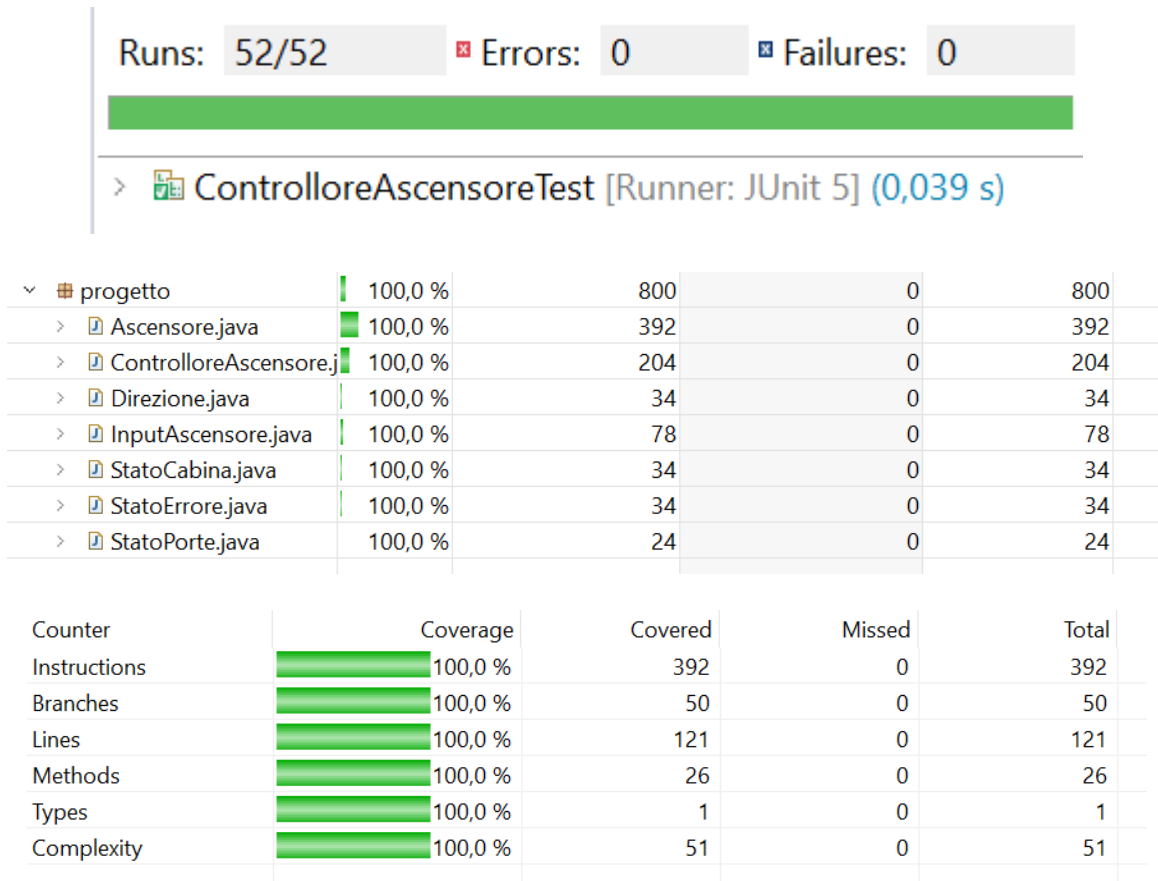
Entrate: 0Uscite: 0Aggiorna persone

12. Test JUnit e Test Parametrici

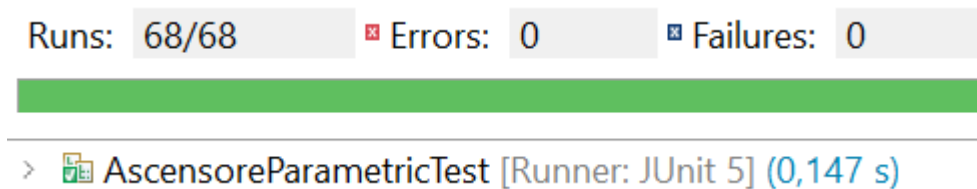
Oltre alla verifica statica tramite JML/OpenJML, il progetto include test JUnit 5 per verificare il comportamento eseguibile della parte Java.

I test sono stati eseguiti sia in ambiente *Eclipse*, sia automaticamente nella pipeline GitHub Actions e la copertura del codice è stata analizzata tramite *EclEmma*.

La classe *ControlloreAscensoreTest* contiene test funzionali sul comportamento del sistema.



La classe *AscensoreParametricTest* contiene invece test parametrici JUnit 5. Questi test permettono di eseguire la stessa logica di verifica su più valori di input, riducendo duplicazioni e rendendo più sistematico il controllo dei casi limite.



12.1 Collegamento tra scenari AVALLA e test JUnit

La seguente tabella riassume la corrispondenza principale tra gli scenari AVALLA manuali e i test JUnit implementati.

Scenario AVALLA	Test JUnit	Verifica
scenario_idle.avalla	idleSenzaRichieste()	Stato iniziale stabile senza richieste
scenario_richiesta_piano_superiore.avalla	richiestaInternaVersoPianoSuperiore(); ascensoreServePianoSuperioreRichiesto()	Richiesta superiore, movimento SU e servizio piano
scenario_richiesta_piano_inferiore.avalla	richiestaVersoPianoInferiore()	Richiesta inferiore e movimento GIU
scenario_overload.avalla	entraInOverload()	Ingresso nello stato OVERLOAD
scenario_risoluzione_overload.avalla	risolveOverloadQuandoUnaPersonaEsce()	Uscita da OVERLOAD
scenario_richieste_durante_overload.avalla	richiesteDuranteOverloadVengonoAcquisite()	Acquisizione richieste durante OVERLOAD
scenario_guasto.avalla	attivaGuastoTecnico()	Ingresso nello stato GUASTO
scenario_blocco_richieste_durante_guasto.avalla	nonAcquisisceNuoveRichiesteDuranteGuasto()	Blocco nuove richieste durante GUASTO
scenario_conservazione_richieste_guasto.avalla	conservaRichiesteGiaAcquisiteDuranteGuasto()	Conservazione richieste già acquisite
scenario_ripristino_guasto.avalla	decrementaTimerDuranteGuasto(); ripristinaDopoGuasto()	Decremento timer e ripristino operativo

12.2 Conversione test Avalla creati con ATGT in JUnit

Oltre agli scenari AVALLA scritti manualmente, sono stati considerati anche alcuni scenari generati automaticamente tramite ATGT.

La conversione è stata effettuata traducendo ogni step dello scenario AVALLA in una chiamata al metodo eseguiPasso() del controllore. Dopo ogni passo sono stati inseriti assert JUnit per verificare lo stato atteso dell'ascensore.

In alcuni casi la conversione è stata adattata:

- Il modello ASM permette di avere più input dello stesso tipo contemporaneamente, mentre la classe Java InputAscensore rappresenta un solo valore per ogni tipo di richiesta. Per questo motivo, alcune richieste sono state precaricate direttamente nello stato dell'ascensore.
- Alcuni scenari contenevano valori numerici estremi per personeEntrate e personeUscite, non rappresentabili in modo significativo nel modello Java basato sul tipo int. In questi casi i test sono stati adattati mantenendo il comportamento logico dello scenario.

13. Test Selenium

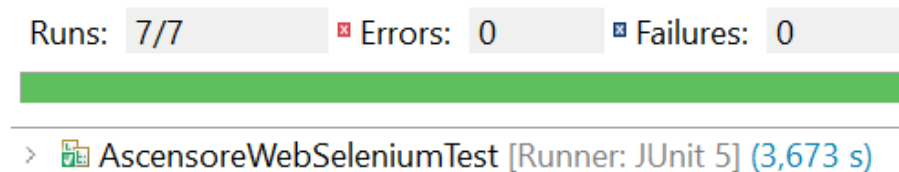
Oltre ai test JUnit sul nucleo logico, il progetto include test Selenium per verificare il corretto funzionamento dell'interfaccia web dimostrativa.

I test Selenium sono test di integrazione, perché verificano il collegamento tra:

- pagina web
- server HTTP
- ControlloreAscensore
- Ascensore

Prima di ogni test viene eseguito un reset dello stato del sistema, così da rendere i test indipendenti tra loro.

Al termine dell'esecuzione, il browser viene chiuso e il server viene arrestato.



```
Server avviato su http://localhost:8080
Starting ChromeDriver 148.0.7778.96 (8625e066febc721e015ea99842da12901eb7ed73-refs/branch-heads/7778@{#1904}) on port 33687
Only local connections are allowed.
Please see https://chromedriver.chromium.org/security-considerations for suggestions on keeping ChromeDriver safe.
ChromeDriver was started successfully on port 33687.
mag 28, 2026 7:47:17 PM org.openqa.selenium.remote.ProtocolHandshake createSession
INFORMAZIONI: Detected dialect: W3C
```

In locale i test utilizzano il ChromeDriver presente nella cartella drivers, mentre nella pipeline GitHub Actions ChromeDriver viene configurato automaticamente per l'ambiente Linux del runner (in CI i test vengono eseguiti in modalità headless, cioè con apertura del browser in background).

14. Verifica tramite ESC/OpenJML

La verifica statica è stata eseguita mediante ESC di OpenJML.

L'obiettivo della verifica è controllare che i metodi della classe rispettino le specifiche JML definite tramite:

- invarianti di classe;
- precondizioni;
- postcondizioni;
- annotazioni sui cicli;
- metodi pure.

La verifica ha permesso di controllare la coerenza del codice rispetto alle proprietà dichiarate e di rafforzare la gestione degli stati di errore.

```

  progetto
  progetto.Ascensore
    [VALID] aggiornaPersone(int,int) [1,423 z3_4_3]
    [VALID] aggiungiChiamataDiscesa(int) [1,637 z3_4_3]
    [VALID] aggiungiChiamataSalita(int) [1,970 z3_4_3]
    [VALID] aggiungiRichiestaInterna(int) [1,889 z3_4_3]
    [VALID] Ascensore() [1,402 z3_4_3]
    [VALID] attivaGuasto() [1,639 z3_4_3]
    [VALID] chiudiPorte() [1,448 z3_4_3]
    [VALID] esisteRichiesta() [1,375 z3_4_3]
    [VALID] esisteRichiestaSopra() [1,662 z3_4_3]
    [VALID] esisteRichiestaSotto() [1,615 z3_4_3]
    [VALID] gestisciSovraccarico() [1,886 z3_4_3]
    [VALID] gestisciTimerGuasto() [1,493 z3_4_3]
    [VALID] getDirezione() [1,460 z3_4_3]
    [VALID] getNumeroPersone() [1,465 z3_4_3]
    [VALID] getPianoCorrente() [1,466 z3_4_3]
    [VALID] getStatoCabina() [1,445 z3_4_3]
    [VALID] getStatoErrore() [1,459 z3_4_3]
    [VALID] getStatoPorte() [1,455 z3_4_3]
    [VALID] getTimer() [1,455 z3_4_3]
    [VALID] indiceDelPiano(int) [1,402 z3_4_3]
    [VALID] mettilInAttesa() [1,579 z3_4_3]
    [VALID] muoviDiUnPiano() [1,541 z3_4_3]
    [VALID] pianoValido(int) [1,479 z3_4_3]
    [VALID] richiestaAttiva(int) [1,809 z3_4_3]
    [INVALID] scegliDirezione() [2,944 z3_4_3]
    [VALID] serviPianoCorrente() [1,754 z3_4_3]
```

15. Conclusione

L'implementazione Java della classe Ascensore rappresenta il nucleo logico principale del sistema ascensore modellato in ASM.

La specifica JML permette di formalizzare le principali proprietà di sicurezza e coerenza dello stato, tra cui la validità del piano corrente, la validità del timer, la non negatività del numero di persone, l'assenza di valori nulli negli stati, il movimento solo con porte chiuse e la corretta gestione degli stati di guasto e sovraccarico.

Il codice Java non sostituisce il modello ASM completo, ma ne implementa una parte significativa in forma eseguibile e verificabile.

La verifica tramite ESC/OpenJML controlla il rispetto delle specifiche JML sui metodi principali della classe Ascensore. I test JUnit 5 verificano invece il comportamento operativo del sistema nei principali scenari previsti, mentre i test parametrici rafforzano il controllo su classi di input, valori limite e condizioni di validità.

Il collegamento tra scenari AVALLA e test JUnit mostra la continuità tra validazione del modello ASM e verifica dell'implementazione Java: alcuni comportamenti validati nel modello astratto sono stati ripresi anche nei test eseguibili del nucleo Java.

L'interfaccia web dimostrativa permette di osservare dinamicamente il comportamento del nucleo Java e di interagire con il sistema in modalità automatica o manuale. Poiché utilizza direttamente Ascensore, ControlloreAscensore e InputAscensore, essa rimane coerente con la logica implementativa verificata.

I test Selenium completano questa parte controllando che pagina web, server HTTP e nucleo logico Java interagiscano correttamente.

Infine, i test automatici JUnit 5, i test parametrici e i test Selenium sono stati integrati in un workflow GitHub Actions, così da automatizzare la compilazione del progetto e l'esecuzione dei test a ogni modifica del repository.

Nel complesso, la parte Java/JML può essere considerata coerente con il modello formale e adeguata come nucleo implementativo del progetto.